项目目录结构文档

项目目录结构文档

本页面整理了 agriops-copilot 项目的完整目录结构,帮助开发者快速理解项目的组成部分、各服务的职责以及代码组织方式。该结构适用于多服务协作架构(multi-service architecture),并支持前后端分离、AI 服务独立运行、数据库集中管理等特性。

📁 顶层目录结构

```
agriops-copilot/
├── java-service/ # Java 后端服务(Spring Boot 3)
├── python-ai/ # Python AI 服务(FastAPI + RAG + Agent)
├── web/ # 前端(Vue3 + Vite)
├── docs/ # 文档目录(可选)
├── compose.yaml # Docker Compose 编排文件
├── .env.example # 环境变量示例文件
└── README.md # 项目说明文档
```

🧱 java-service(Spring Boot 后端)

```
java-service/
├── src/
│ ├── main/
│ │ ├── java/ # Java 源代码
│ │ ├── resources/ # 配置文件(application.yml 等)
│ └── test/ # 单元测试
├── pom.xml # Maven 构建文件
└── Dockerfile # Java 服务 Docker 构建文件
```

📌 主要职责

- 权限管理(RBAC)
- 审批流(Workflow)
- 业务数据接口(库存、采购、供应链)
- 与 Python AI 服务通信
- SSE 流式输出支持

🐍 python-ai(AI 服务)

```
python-ai/
├── app/
│   ├── api/ # FastAPI 路由
│   ├── rag/ # RAG(向量检索)
│   ├── agents/ # Agent 工具调用逻辑
│   ├── models/ # 模型封装(OpenAI / Ollama)
│   └── utils/ # 工具类
├── requirements.txt # Python 依赖
└── Dockerfile # Python 服务 Docker 构建文件
```

📌 主要职责

- RAG(检索增强生成)
- Agent 工具调用
- 文档解析(PDF、Word、Excel)
- 与 Java 服务协作处理业务逻辑

🎨 web(前端 Vue3)

```
web/
├── src/
│   ├── components/ # 组件
│   ├── pages/ # 页面
│   ├── stores/ # Pinia 状态管理
│   ├── api/ # 后端接口封装
│   └── utils/ # 工具类
├── public/ # 静态资源
├── package.json # 前端依赖
└── vite.config.js # Vite 配置
```

📌 主要职责

- 用户界面(UI)
- SSE 流式输出展示
- 引用(RAG)展示
- 草稿确认界面
- 与 Java / Python 服务交互

🗄️ compose.yaml(Docker 编排)

用于一键启动整个系统:

```
services:
  java-service:
  python-ai:
  web:
  postgres:
```

📌 主要职责

- 启动所有服务
- 配置网络
- 配置环境变量
- 挂载数据卷

🧩 .env.example(环境变量示例)

包含:

- 数据库连接
- AI 服务地址
- Java 服务端口
- 前端环境变量

开发者复制为 .env 使用。

📚 README.md

项目总说明,包括:

- 项目介绍
- 启动方式
- 文档链接
- 开发规范

本页面为 agriops-copilot 的标准目录结构说明,可作为所有开发者的参考基础。